Working with AI: From Prompting to Engineering

I am increasingly treating AI not as a tool that I give instructions to, but as a participant in an engineered software-development process. The distinction matters. A conventional software tool is generally deterministic enough that its interface can be understood in terms of inputs, outputs and side effects. An LLM is not. It is capable of reasoning, synthesis and implementation, and it applies those capabilities to whatever its operator supplies, including what the operator did not mean to supply.

A prompt is usually imagined as an instruction sent to a model: do X. That is not what I am describing here. What I am describing is closer to the design of session types — units of work with preconditions, permitted operations, constraints, participants, artefacts and an expected transition in the state of the work — in which an LLM happens to be the mechanism that performs the transition. The rest of this essay is an account of why that framing is necessary and what it looks like in practice.

That structure is not there to constrain the model. It is there to discipline the person using it, above all where the work is done inside an organisation, on legacy systems or on proprietary vendor platforms.

The hazard is not that the model is unreliable. It is that the model is very good at three things. It reads a prompt between the lines, so the operator's assumptions and preferred solution find their way into the output whether or

not they were stated. It infers from patterns, and in a legacy system it has no means of telling which patterns are legitimate. And where it lacks knowledge, as it does of any proprietary platform that formed no part of its training, it supplies something plausible in place of the gap.

Each of these makes the agent an amplifier of whatever its operator brings to the session. An individual developer working this way can demonstrate an increase in productive output, and the increase is real. The trap lies at the level of the organisation. Many developers, each leading the agent differently, each with different solution preferences and different gaps in what they know, are all producing plausible work at speed. That work does not converge. What is being accelerated is divergence, and the resulting incoherence is a risk that no individual's output will reveal.

My response has therefore been to put structure around the interaction. What the framework says, in effect, is: "This is the process. This is how we work. These are the demands at each stage. This is the tacit and proprietary knowledge you need." Because those demands are explicit, the agent can hold the operator to them. It can say when a session falls short of what is required, and it can say when the operator is attempting to hand over a judgement, or the ownership of an output, that remains theirs.

The system I have been developing began as a set of practices around an LLM-assisted software project. The project contains the familiar artefacts of software development — requirements, user stories, architectural decisions,

technical investigations, tasks and code — but the interesting part is the relationship between those artefacts and the AI sessions that work on them. What emerged was effectively a small language for software development: different sessions have different purposes, different authorities and different permissible outputs.

The central idea is the role. A role defines what kind of work a session is allowed to perform. It establishes the form of the interaction, its constraints and the conditions under which the session can begin. It also defines what constitutes a successful output. Those conditions are addressed to the operator as much as to the agent: where they have not been met, the agent is expected to say so rather than proceed.

For example, requirements establishment is not implementation. It is an interview process in which one or more interested parties — the project's personas — establish what is valuable and what is actually wanted. The output is an intentionally coarse item of work. Refinement subsequently decomposes that into independently deliverable pieces. Architectural analysis asks a different question: how does the requirement fit the existing estate, and what does it touch? Technical design is conditional; it is needed when the implementation problem is genuinely new rather than merely unfamiliar. A Three Amigos session — the agile practice in which business, development and testing perspectives examine a story together — acts as a gate, checking that

the relevant concerns have been considered before development begins. Finally, a TDD implementation session writes the software.

This is superficially similar to a conventional development workflow, but the AI changes the significance of the boundaries. An LLM is extremely good at moving forward. Given an ambiguous requirement, it will happily resolve the ambiguity, usually in whichever direction the operator's phrasing leans. Given a technical question encountered during implementation, it will often answer it. Given an opportunity for abstraction, it may create one. These behaviours are useful when they are wanted and dangerous when they are not. In each case a decision has been taken, and without a boundary the operator has neither made it nor seen it made.

Consequently, the framework treats not knowing as a legitimate state.

A technical question discovered while implementing a story does not necessarily become part of the implementation. It can be recorded as a finding and deferred. An architectural uncertainty can become an ADR. A question that requires investigation can become a spike. A task can remain a task rather than expanding into domain modelling. A code review can identify an architectural concern without turning the review into an architectural redesign.

This preserves the provenance of knowledge. The important distinction is not simply between "done" and "not done", but between what we know, what we have decided, what we want, and what we have yet to establish.

The personas are part of the same mechanism. They are not primarily attempts to make the AI role-play a customer or an operations engineer. They describe the interests, constraints and failure modes of particular parties in the system. A customer persona establishes a perspective from which customer outcomes matter. An operations persona brings concerns such as scheduled processing, dependency health, observability, exception handling and the dangers of partial processing. These perspectives can be brought into a requirements session when they are relevant, and they do not always agree: the customer persona wants an order to complete; the operations persona wants an order that cannot complete to fail loudly, early and recoverably. A requirement that satisfies only one of them is incomplete.

This gives the AI something more useful than a generic instruction to "consider the user". It gives it competing interests to reason about, and it ensures that the operator's own preferences are not the only interests present in the session.

The legacy code problem

This approach becomes particularly interesting when the system being worked on is not new.

With greenfield code, an agent can often infer a reasonable amount of the intended architecture from the requirements, the surrounding conventions and the code it has just produced. Legacy code is different. It contains history. Some patterns are deliberate; some are accidents; some were once deliberate

and have since become liabilities. Some exist because of constraints that are no longer visible. Others exist simply because nobody has yet had sufficient reason to remove them.

An LLM presents a particular danger here because it is extremely good at recognising patterns.

Given a sufficiently large codebase, an agent can quickly discover that certain structures, naming conventions or implementation techniques occur repeatedly. It is tempting to assume that this constitutes architectural knowledge. It may not. The agent can mistake prevalence for legitimacy.

A pattern that appears fifty times in a legacy system may therefore be reinforced rather than challenged. The agent sees evidence that "this is how we do things" when the experienced developer sees evidence of a historical problem that needs to be unwound.

A concrete case. Consider a small service that replaces a pair of legacy integration components, each carrying messages in one direction between two platforms. The code of the two components presents itself as a convention — one service per direction, each structured the same way — and an agent reading the estate would reasonably reproduce that convention. It is not a convention. It is an accident of how the services were commissioned, and the replacement deliberately collapses the two directions into one service. The legacy code also contains a subtler trap: one of the components silently fails to recover its connection after routine maintenance on the platform it reads

from, and messages stop flowing until someone notices. Nothing in the code marks this as a defect; the failure is swallowed, and to a pattern-matching reader the swallowing looks like a tolerated idiom rather than a bug with an operational history. Finally, there is knowledge the codebase cannot contain at all: while old and new run side by side, the new service must remain compatible with a setting held by the legacy deployment, because the constraint lives in the runtime environment, not in any repository. An agent working from the code alone would either miss that requirement or invent a rationalisation for it. A developer new to the estate is in the same position, and with an agent that developer reaches the wrong conclusion faster and with a more convincing account of it.

The problem in each case is not that the agent has failed to comprehend the code. It may have comprehended it remarkably well. The problem is that comprehension and judgement are different activities.

I addressed this with another role: the cold reader.

The cold reader is deliberately given a different task from an implementation agent. It is unleashed on the codebase with the instruction to attempt to comprehend it and to describe that comprehension. It is asked to identify ambiguities, suspect patterns and outright bugs. It is not initially asked to fix anything.

That distinction is important. The objective is to produce an explicit model of what an intelligent but uninformed observer can infer from the code itself.

The resulting reading is then subjected to a senior-developer interview process. The purpose of the interview is to surface the knowledge that exists around the code but is not encoded in it: historical reasons, external constraints, known weaknesses, intended strategic direction and the accumulated judgement of people who have lived with the system. In the case above, the interview is where the two-service split is identified as an accident, the recovery failure as a defect with a known trigger, and the compatibility setting as a temporary constraint with a planned expiry.

The two perspectives are complementary. The cold reader provides fresh observation without institutional memory. The senior developer provides institutional memory without the illusion that the existing code is necessarily correct.

Their combination allows me to construct something more useful than either alone: a corpus of documentation describing refactoring strategies, engineering standards and the direction in which the system should move.

That corpus then becomes part of the context for subsequent coding sessions.

This closes an important loop. The legacy code initially contains a mixture of good practice, historical accident and accumulated compromise. An agent looking only at the code cannot reliably distinguish those categories because

the code does not contain all the information necessary to make the distinction. The cold reading exposes what can be inferred; the interview supplies what cannot; the resulting corpus explicitly identifies the difference.

The next implementation agent therefore does not have to infer engineering standards from the statistical distribution of patterns in the legacy code. It can be told which patterns are authoritative, which are tolerated, which are suspect and which are actively being removed. Nor does each developer have to supply those standards from memory or preference. Every session starts from the same account of the system, whoever is running it.

In effect, I am extracting a missing semantic layer from the system and then feeding it back into the development process.

The same applies, more sharply, to proprietary vendor platforms. Legacy code can at least be read. A vendor platform's internals are closed, its behaviour is learned through experience of running it, and it formed no part of the model's training. Asked about it, the agent does not report a gap; it papers over the gap with an answer assembled from platforms that resemble it. An operator who does not know the platform cannot tell the difference, and one who does may not think to check. The remedy is the same: what the organisation knows about the platform is written into the corpus, and a session that depends on that knowledge and does not have it should not begin.

Both are instances of the same underlying principle: when the available knowledge is insufficient for safe action, create a session whose purpose is to

establish the missing knowledge rather than allowing the agent to invent it or the operator to assume it.

From prompting to session design

There is another layer to this approach: the model is not treated as the sole source of procedural authority.

Different models have different native instruction mechanisms and different behavioural tendencies. In particular, I have found that the surrounding environment matters considerably. Claude, for example, gives substantial weight to its CLAUDE.md configuration. I use this to establish engineering invariants such as TDD, KISS, DRY, avoidance of speculative generality, and explicit resistance to scope creep.

That configuration is deliberately different from the workflow framework.

The framework describes what kind of work is being done. The model-specific configuration describes how the engineering collaborator should behave while doing it. The former is intended to be agent-neutral; the latter necessarily acknowledges the characteristics of a particular agent. This separation is important because otherwise the process becomes coupled to whichever model happens to be fashionable.

My own engineering instructions also contain a philosophical component: an ontology concerned with distinctions such as the difference between a thing and a record of a thing, and the temporal relationship between a state and an

observation of that state. In an order-fulfilment system these distinctions are load-bearing. An order, the record of an order, and an observation that an order has been fulfilled are three different things with three different temporal characters, and a model that flattens them into a generic "order object" will design reconciliation wrongly. The ontology gives the AI the vocabulary to keep them apart, and it is explicitly discouraged from collapsing them into generic engineering language.

This also explains why I am interested in keeping the framework agent-neutral. If the essential process can only be expressed as "tell Claude to behave like this", then the abstraction is probably too close to the implementation. If instead I can describe the process as transformations between engineering states — requirements becoming refined stories, uncertain architectural questions becoming decisions, legacy code becoming documented engineering direction, ready stories becoming tested code — then an LLM is an implementation detail of the process.

The AI is consequently neither an autonomous developer nor merely autocomplete. It is closer to a reasoning component inside an explicit engineering process, one that knows what the process demands of the person operating it.

That distinction changes where responsibility sits. I do not try to make the model trustworthy by giving it a sufficiently elaborate personality, and I do not try to make the operator disciplined by exhortation. The process states what

each session requires, and the agent is expected to say when a requirement has not been met: a precondition missing, a decision left unmade, a judgement the operator is trying to hand over. I use gates where crossing from one epistemic state to another matters. I separate exploration from commitment. I make discoveries visible rather than allowing them to be silently absorbed into whatever the session happens to be doing. The operator remains the owner of what the session produces, and the framework makes that ownership difficult to disown.

None of this is enforcement. The operator can override any gate by a simple claim to authority, and the agent will give way even after it has objected. That is a limitation of how LLMs currently work, and I do not see a way of preventing it. The framework therefore depends on a degree of trust, and the word for what it does is discipline, not constraint. The framework states what the work requires and the agent says when a session falls short of it. What happens next is the operator's decision.

A fair objection is that this is a great deal of ceremony to put around a developer and a text predictor. It is, and the ceremony is not free; a weekend script does not need a Three Amigos gate. The framework pays for itself where many people work on the same estate and the cost of a confidently wrong transition is high: regulated domains, irreversible operations, proprietary platforms, legacy estates whose constraints live outside the code. In those

settings the ceremony is not overhead added to the engineering. It is the engineering.

The interesting unit of engineering is therefore no longer necessarily the prompt or even the model. It is the interaction between an operator, a model, a defined role, a body of artefacts, and a controlled transition in the state of the work.

That is how I am working with AI: not by asking it to replace the development process, but by making the development process explicit enough that an AI can participate in it and hold its operator to what the process requires.